7.2 回退链与性价比优化

> **生成模型**：GPT-5.4 (openai/gpt-5.4)
> **Token 消耗**：输入 ~18,000 tokens，输出 ~3,900 tokens（本节）

如果说上一节解决的是“模型怎么接进来”，这一节解决的就是“接进来以后，怎么让它跑得稳、跑得值”。很多人一开始只盯着模型能力，最后账单一出来，人先清醒了。OpenClaw 这套模型层真正厉害的地方，不只是能切模型，而是能把稳定性和成本一起纳入配置里。

你可以把这一节记成一句话：**主模型负责质量，回退链负责生存，便宜模型负责把账单打下来。**

7.2.1 回退链到底是什么

回退链不是负载均衡，也不是随机抽签。它的逻辑很简单：先试主模型，失败了再按顺序试后备模型。

这里的“失败”可能包括很多情况：

- API 限流
- 认证失效
- 余额耗尽
- 提供者暂时不可用
- 当前模型不支持所需能力
- 上下文窗口不够

从概念上看，OpenClaw 的兜底通常有三层：

1. 主模型
2. 回退模型列表
3. 提供者内部的认证轮换

注意顺序别记反。对使用者来说，你最关心的是“主模型失败以后，下一跳是谁”；对系统实现来说，它通常还会先在同一提供者内部尝试别的认证档案，再决定要不要切换到别的模型。

7.2.2 一个最常见也最实用的回退结构

很多人最后都会落到类似这样的结构：

```
高质量主模型 -> 同档次备份模型 -> 便宜保底模型 -> 本地应急模型
```

翻成实际配置，大概就是：

```
{
  agents: {
    defaults: {
      model: {
        primary: "anthropic/claude-sonnet-4-6",
        fallbacks: [
          "openai/gpt-4o",
          "google/gemini-2.5-pro",
          "openrouter/meta-llama/free",
          "local/qwen2.5-coder-14b"
        ]
      }
    }
  }
}
```

这个顺序不是唯一答案，但思路很重要。

第一跳保质量。Claude 出问题时，不要直接掉到太弱的模型，不然用户会明显感觉“今天这 AI 怎么突然变笨了”。

第二跳保可用。Gemini 这种覆盖面广、成本相对平衡的模型，很适合当跨提供者的中间保险。

第三跳保成本。OpenRouter 上的免费模型或者超低价模型，在“只要能答就行”的场景里很有价值。

第四跳保生存。本地模型哪怕效果一般，至少在外网抽风时还能维持基本可用。

7.2.3 不同任务，主模型应该不一样

一个常见误区是：我找到了最强模型，于是所有事都让它做。这样最容易把钱烧掉。

更合理的做法是按任务类型分层：

任务类型	推荐主模型策略	为什么
复杂代码、架构分析	高质量模型	推理深度和稳定性更重要
日常问答、摘要、改写	中价或低价模型	这类任务对极致能力没那么敏感
批量处理、流水线生成	便宜模型优先	单次效果差一点，总成本差很多
隐私敏感、本地文档处理	本地模型或私有网关	可控性比绝对质量更关键

说得直白点：不是所有 prompt 都配得上最贵模型。

7.2.4 什么时候该上贵模型，什么时候别硬上

最值得花钱的，通常是下面几类任务：

- 一次出错代价很高，比如改核心代码、写数据库迁移、生成生产配置
- 需要长链路推理，比如排查复杂 bug、做跨模块设计
- 上下文很长，历史对话和工具结果非常多
- 结果质量差一点就会引发连锁返工

不太值得一上来就砸贵模型的任务，也很典型：

- 翻译、润色、格式改写
- 从日志里抽几条关键信息
- 对几十份相似文本做批量摘要
- 生成模板化内容初稿

这里有个很实用的判断方法：**如果模型输出之后，你本来就准备再人工过一遍，那就先别急着上最贵的。**

7.2.5 性价比优化，不只是换便宜模型

很多人一说成本优化，脑子里只有一件事：换便宜模型。其实还不够。

真正的成本优化，至少有四种手段。

第一，分层用模。
高质量模型只留给高价值任务，别拿它做低门槛流水活。

第二，控制上下文长度。
上下文越长，成本越高。尤其是 Agent 场景里，工具输出、历史对话、系统提示会把 token 撑得很快。该裁剪的就裁剪，该总结的就总结。

第三，给回退链分档。
不要把三个同价位的昂贵模型串成回退链，那不叫优化，只是换品牌继续烧钱。

第四，利用免费和低价入口做扫描与预筛。
比如 OpenRouter 上经常能找到免费试用模型，虽然不适合做主力，但拿来做冷启动、实验、备份，非常划算。

7.2.6 OpenRouter 免费模型扫描怎么用

OpenRouter 对 OpenClaw 用户很有吸引力的一点，是它把很多不同提供者的模型聚合在一起，而且经常会有免费模型或限免档。OpenClaw 也考虑到了这个场景，可以直接扫描可用模型。

常见命令：

```
openclaw models scan
openclaw models scan --no-probe
openclaw models scan --set-default
```

`scan` 的意义，不只是“看看有啥模型”，更实际的用途有两个。

一个是冷启动。你刚装好 OpenClaw，还没想清楚长期方案，可以先扫一遍，找一个能跑的免费模型，把整个系统先带起来。

另一个是低成本备份。主回路用稳定付费模型，回退链尾部挂一个 OpenRouter 免费模型，关键时刻至少不会完全失明。

一个典型配置可以这样写：

```
{
  agents: {
    defaults: {
      model: {
        primary: "anthropic/claude-sonnet-4-6",
        fallbacks: [
          "google/gemini-2.5-pro",
          "openrouter/meta-llama/free"
        ]
      }
    }
  },
  models: {
    providers: {
      openrouter: {
        apiKey: "$OPENROUTER_API_KEY"
      }
    }
  }
}
```

当然，免费模型有代价：速度、稳定性、上下文能力和结果质量，通常都不如主流付费型号。所以它更像“保命包”，不是全能答案。

7.2.7 一个够用的性能对比表

下面这张表不是实验室精确测评，而是从 OpenClaw 使用者的角度，给你一个选型感觉。评分是相对值，重点看思路，不要把分数当真理。

模型档位	代表模型	质量	速度	成本	适合做什么
高质量旗舰	Claude Sonnet / GPT-4o 级	9	7	3	复杂代码、长链推理、关键任务
平衡型	Gemini Pro / 中高档国产模型	8	8	6	日常主力、回退、长文本处理
轻量型	mini/flash/small 档	6	9	9	摘要、改写、分类、批处理

模型档位	代表模型	质量	速度	成本	适合做什么
免费/ 低价聚合	OpenRouter free 类	4-6	6	10	冷启动、试水、尾部兜底
本地模型	Ollama/Qwen/Llama 本地部署	5-8	5-8	10	隐私、本地化、 无外网兜底

这里的“成本”分越高，表示越省钱。

从这张表里你能看出一件事：真正划算的配置，不是把所有请求都压到同一类模型，而是让不同层级干不同的活。

7.2.8 回退链的顺序怎么排才合理

一个好用的回退链，一般遵循三条原则。

第一，先保体验一致。
主模型失败后的第一回退，最好还是同等级能力的模型。这样用户感知最小。

第二，再保成本可控。
第二或第三回退可以逐渐下探到更便宜的模型。因为走到这一步，本来就是异常路径了。

第三，最后保绝对可用。
尾部可以挂免费模型或本地模型，哪怕质量一般，至少系统不至于完全瘫掉。

反过来说，不推荐这样排：

```
最强模型 -> 更贵模型 -> 还是贵模型 -> 继续贵模型
```

这种配置看上去很豪华，实际上对成本一点都不友好，对可用性提升也不见得成比例。

7.2.9 三种很实战的配置模板

模板 A：质量优先

适合自己写代码、做研究、日常高强度使用。

```
{
  agents: {
    defaults: {
      model: {
        primary: "anthropic/claude-sonnet-4-6",
        fallbacks: ["openai/gpt-4o", "google/gemini-2.5-pro"]
      }
    }
  }
}
```

特点是体验稳，但成本不低。

模板 B：平衡优先

适合大多数个人用户。

```
{
  agents: {
    defaults: {
      model: {
        primary: "google/gemini-2.5-pro",
        fallbacks: ["anthropic/claude-sonnet-4-6", "openrouter/meta-llama/free"]
      }
    }
  }
}
```

这个方案没那么极致，但价格和体验通常更均衡。

模板 C：预算优先

适合 VPS 小机器、学生党、实验环境。

```
{
  agents: {
    defaults: {
      model: {
        primary: "openrouter/meta-llama/free",
        fallbacks: ["local/qwen2.5-coder-14b", "google/gemini-2.5-flash"]
      }
    }
  }
}
```

这个配置不追求豪华，核心目标是“先跑起来，再逐步升级”。

7.2.10 中国用户做性价比优化时要额外考虑什么

对国内用户来说，成本优化不能只看 API 单价，还得把网络条件算进去。

比如某个国际模型单价看起来不贵，但你本地网络经常超时，结果反复重试、回退、等待，实际体验很差；反过来，一个国产模型单价未必最低，但延迟低、成功率高，综合下来反而更值。

所以中国用户选型时，最好同时看四件事：

- 单价
- 延迟
- 稳定性
- 网络可达性

这也是为什么不少人最后的主力方案会变成：国内直连模型做主模型，国际模型做高质量回退。

7.2.11 别把“回退成功”当成系统没问题

最后提醒一个容易忽略的点：回退链的目标是提高可用性，不是掩盖问题。

如果你的系统天天都在回退，那不是说明设计得好，而是说明主模型、认证、账单或者网络已经有问题了。回退应该是保险丝，不是常态。

真正健康的状态应该是：

- 主模型承担大部分流量
- 第一回退偶尔触发

- 第二回退很少触发
- 免费模型或本地模型几乎只是应急

如果顺序反过来，说明你该回去改配置，而不是夸自己“容错做得真棒”。

本节小结

回退链的核心，不是简单地多挂几个模型，而是给模型分工：主模型保质量，第一回退保体验，后续回退保成本和生存。真正的性价比优化，也不是无脑换便宜模型，而是把任务分层、上下文控住、免费模型用在该用的地方。对中国用户来说，还要把网络可达性一起算进去。不然纸面价格看着香，跑起来一点都不香。